

ECOM EXPERTS

Ecom Adblocker para Firefox

Manual de usuario e instalación

ecom_adblocker-firefox 1.3.1 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

Para probarla, sin firmar

Para uso normal

Qué crea y qué pasa al desinstalar

Permisos: lo que Firefox te va a preguntar

4. Configuración

El interruptor general

Los dos contadores

Permitir en este sitio

Las nueve listas

Permitir anuncios en un canal de YouTube

5. Cómo se usa, día a día

6. Qué se ve en las páginas

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que se ha arreglado

Lo que del arreglo de Chrome NO se ha traído, y por qué

Lo que se ha revisado y estaba bien

Lo que queda pendiente

9. Ficha técnica

Estructura

Permisos y por qué

Guiones de contenido

Mensajes internos

Claves de almacenamiento

Reglas dinámicas

Validación

Banco de pruebas

Registros

Versión 1.3.1 · Ecom Experts · Extensión de navegador (Manifest V3)

1. Qué hace y para quién

Ecom AdBlocker quita la publicidad y los rastreadores de las páginas que visitas. No es un filtro de listas comprado a nadie ni un envoltorio de otro producto: es una extensión propia, con **29.740 reglas de bloqueo de red** y **13.910 selectores de ocultación** metidos dentro del paquete, que funciona sin conectarse a ningún servidor.

Trabaja en tres capas que se complementan. La primera es el **filtrado de red**: cuando la página pide una imagen, un guion o un vídeo a un servidor de publicidad, la petición no llega a salir del navegador. Eso ahorra tiempo de carga y datos, y es lo que impide que los rastreadores construyan un perfil de lo que lees. La segunda es el **filtrado cosmético**: hay huecos que el anuncio deja aunque no se descargue —marcos vacíos, franjas grises, esas imágenes de fondo que visten la página entera con una marca—, y esta capa los oculta. La tercera son los **anuncios dentro del vídeo**, que no viajan por un servidor aparte sino incrustados en el mismo flujo: para YouTube, Disney+, Prime Video y HBO Max hay un guion específico que los salta.

Está pensado para quien quiere navegar limpio sin renunciar al control. Cada una de las listas se enciende y se apaga por separado, cualquier sitio se puede dejar fuera del filtro con un botón, y en YouTube se pueden **permitir los anuncios de un canal concreto** para que el creador siga cobrando por lo que ves. Esa última pieza no es habitual y es deliberada: bloquear no tiene por qué significar dejar sin ingresos a quien te gusta.

No recoge ni un dato. No hay analítica, ni servidores propios, ni cuenta de usuario, ni una sola llamada a Internet en todo el código. Lo único que guarda son tus preferencias, en el almacenamiento local de tu propio navegador.

2. Compatibilidad

Navegador	Firefox 140 o superior (escritorio)
Firefox para Android	142 o superior
Manifest	Versión 3
Identificador	<code>ecom-adblocker@ecomexperts.es</code>
Recogida de datos declarada	Ninguna (<code>data_collection_permissions: none</code>)
Permisos	<code>declarativeNetRequest</code> , <code>storage</code> , <code>activeTab</code>
Dependencias externas	Ninguna. Ni librerías, ni CDN, ni servicios
Idioma de la interfaz	Castellano

El mínimo de Firefox 140 no es un capricho: la extensión declara en el manifest que **no recoge ningún dato**, y esa declaración (`data_collection_permissions`) solo la entiende Firefox a partir de la 140 en escritorio y de la 142 en Android. Con versiones anteriores el validador de Mozilla la rechaza como incompatible.

Esta versión es solo para Firefox. La de Chrome, Edge y Brave es un paquete distinto, con el mismo nombre y las mismas listas pero con diferencias reales en el manifest y en el código. No se pueden intercambiar: Firefox no arranca el proceso de fondo del paquete de Chrome, y Chrome rechaza el de Firefox.

3. Instalación

Para probarla, sin firmar

1. Abre `about:debugging#/runtime/this-firefox`.
2. Pulsa **Cargar complemento temporal**.
3. Elige el fichero `manifest.json` de la carpeta de la extensión.

Se instala al momento, pero **desaparece al cerrar Firefox**. Sirve para probar, no para usarla a diario.

Para uso normal

Firefox de serie **no carga extensiones sin firmar**, así que hay que pasarla por addons.mozilla.org. Con la herramienta `web-ext`:

```
web-ext lint           comprueba que no hay errores
web-ext build          genera el .zip
web-ext sign --channel=unlisted lo firma y devuelve el .xpi instalable
```

El `.xpi` firmado se instala arrastrándolo sobre la ventana de Firefox, o desde `about:addons` → engranaje → **Instalar complemento desde archivo**.

Qué crea y qué pasa al desinstalar

No crea ficheros, ni carpetas, ni bases de datos, ni nada fuera del navegador. Lo único que guarda son estas claves en el almacenamiento local de la extensión:

Clave	Qué es
<code>enabled</code>	El interruptor general
<code>lists</code>	Qué listas están encendidas
<code>whitelist</code>	Los dominios en los que no filtra
<code>ytWhitelist</code>	Los canales de YouTube a los que se les permiten anuncios
<code>totalBlocked</code>	El acumulado del contador

Al desinstalar, Firefox borra ese almacenamiento y las reglas de bloqueo dejan de aplicarse de inmediato. No queda ningún resto.

Permisos: lo que Firefox te va a preguntar

Firefox pedirá **acceso a los datos de todos los sitios web**. Hace falta para el filtrado cosmético, que necesita mirar la página para ocultar los huecos de los anuncios. Es un permiso amplio y conviene entender qué implica: la extensión lee el contenido de las páginas que visitas, pero **no lo envía a ninguna parte**, porque no tiene a dónde enviarlo.

En Firefox ese permiso es **revocable**. Desde `about:addons` → Ecom AdBlocker → **Permisos** puedes quitárselo cuando quieras. Si lo haces, el filtrado de red sigue funcionando (no necesita permisos de sitio), pero **se pierde el filtro cosmético**, porque Firefox deja de inyectar los guiones de contenido.

4. Configuración

Toda la configuración está en la ventana emergente, la que se abre al pulsar el icono de la extensión. No hay página de opciones aparte: lo que ves aquí es todo.

Ecom AdBlocker

1234

en esta pestaña

48.213

total bloqueados

elmundo.es

Permitir en este sitio

LISTAS DE FILTROS ACTIVAS

EasyList 16.000

EasyList Spanish 617

EasyPrivacy 6000

Consent Mode / Google Tags 13

Dominios extra (hosts) 7000

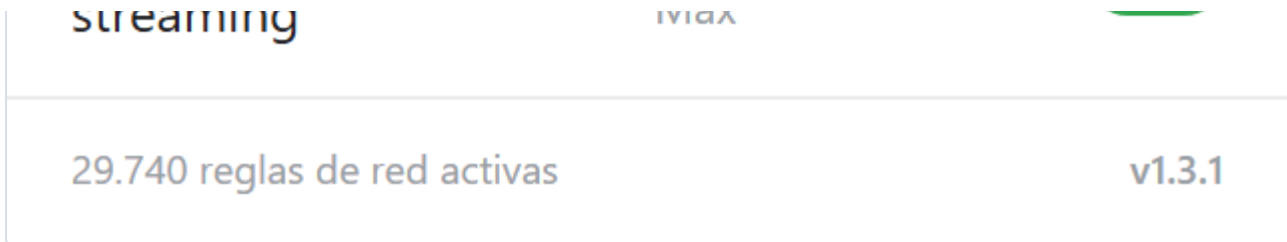
Filtro cosmético 13.910

Anti-anuncios YouTube

Anti-anuncios streaming Disney+ / Prime / Max

Ecom Experts

Pagina 7 de 23



El interruptor general

Arriba a la derecha. Apagado, se desactivan las seis listas de red y los guiones de contenido dejan de ocultar nada. Es el «apágalo todo un momento» de cuando una página no funciona y quieres descartar que sea culpa del bloqueador.

Cuando lo apagas, el pie cambia de «29.740 reglas de red activas» a «**Bloqueo desactivado**», para que no quede duda de que ha surtido efecto.

Ecom AdBlocker

1234

en esta pestaña

48.213

total bloqueados

elmundo.es

Permitir en este sitio

LISTAS DE FILTROS ACTIVAS

EasyList 16.000

EasyList Spanish 617

EasyPrivacy 6000

Consent Mode / Google Tags 13

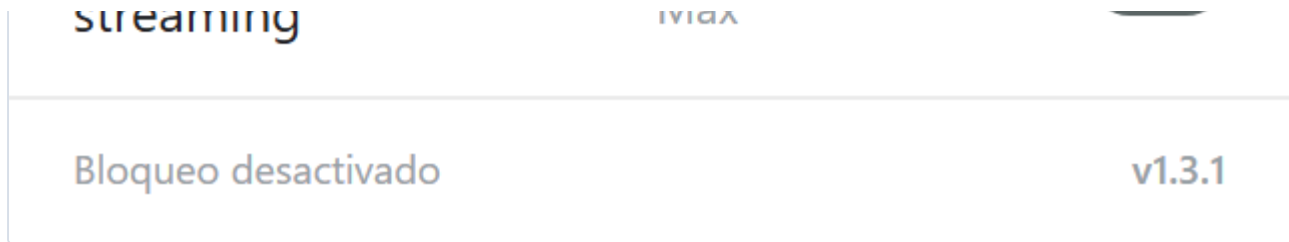
Dominios extra (hosts) 7000

Filtro cosmético 13.910

Anti-anuncios YouTube

Anti-anuncios streaming

Disney+ / Prime / Max



Los dos contadores

«En esta pestaña» y «total bloqueados». **En Firefox muestran un guion**, y no es un fallo: Firefox no permite a las extensiones preguntar qué peticiones ha bloqueado. Está explicado en detalle en el apartado 7. El bloqueo funciona; lo que no se puede es contarlos.

Ecom AdBlocker

en esta pestaña

total bloqueados

elmundo.es

Permitir en este sitio

LISTAS DE FILTROS ACTIVAS

EasyList 16.000

EasyList Spanish 617

EasyPrivacy 6000

Consent Mode / Google Tags 13

Dominios extra (hosts) 7000

Filtro cosmético 13.910

Anti-anuncios YouTube

Anti-anuncios streaming

Disney+ / Prime / Max

streaming	IVIDA	
29.740 reglas de red activas		v1.3.1

Permitir en este sitio

Debajo del dominio de la pestaña. Añade ese dominio a la lista blanca: deja de filtrarse por completo, tanto de red como cosmético. La página se recarga sola para que el cambio se note al momento, y el botón pasa a decir «**Sitio permitido (quitar)**» en verde.

Ecom AdBlocker

1234

en esta pestaña

48.213

total bloqueados

elmundo.es

Sitio permitido (quitar)

LISTAS DE FILTROS ACTIVAS

EasyList 16.000

EasyList Spanish 617

EasyPrivacy 6000

Consent Mode / Google Tags 13

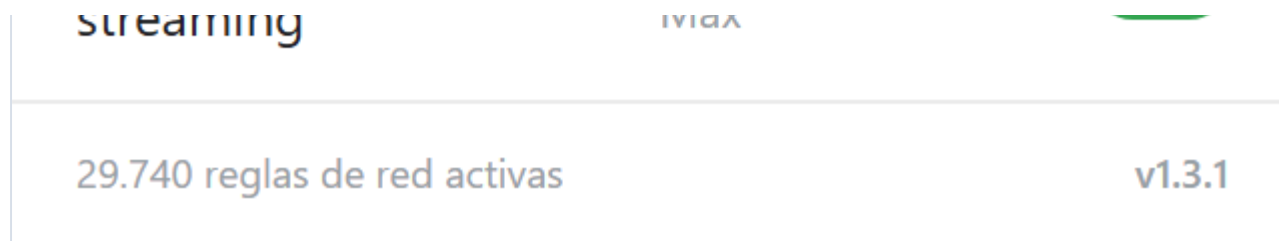
Dominios extra (hosts) 7000

Filtro cosmético 13.910

Anti-anuncios YouTube

Anti-anuncios streaming

Disney+ / Prime / Max



Úsalo cuando una tienda no te deje pagar, un banco no cargue bien o un vídeo no arranque. Es el arreglo de un clic para el 95 % de los problemas.

En pestañas sin dominio — `about:addons` , la página de inicio, un fichero local— el botón sale desactivado y arriba pone «página interna». Es lo correcto: ahí no hay nada que filtrar.

Las nueve listas

Interruptor	Qué apaga	Cuándo tocarlo
EasyList (16.000)	El grueso del bloqueo de publicidad internacional	Casi nunca. Es la lista principal
EasyList Spanish (617)	Redes publicitarias españolas	Casi nunca
EasyPrivacy (6.000)	Rastreadores y analítica	Apágalo si trabajas con analítica web y necesitas ver tus propias etiquetas dispararse
Consent Mode / Google Tags (13)	Google Tag Manager y el modo de consentimiento	Igual que el anterior: apágalo si estás depurando etiquetado
Dominios extra (hosts) (7.000)	Una lista de dominios sueltos de publicidad y malware	Si sospechas que está tumbando un dominio legítimo
Filtro cosmético (13.910)	La ocultación de huecos y el borrado de fondos publicitarios	Si una página se ve descuadrada. Es el sospechoso número uno de los problemas de maquetación
Anti-anuncios YouTube	El guion que salta los anuncios de vídeo	Si YouTube se comporta raro tras un cambio suyo
Anti-anuncios streaming	Los guiones de Disney+, Prime Video y HBO Max	Si una de esas plataformas falla al reproducir

El pie de la ventana suma en vivo las reglas de red que quedan activas, así que ves el efecto de cada interruptor al momento.

Permitir anuncios en un canal de YouTube

Estando en YouTube aparece un botón más, con el nombre del canal que estás viendo. Al pulsarlo, **ese canal deja de tener los anuncios bloqueados**. Es para apoyar a los creadores que te importan sin desactivar el bloqueador en todo YouTube. Los canales permitidos salen luego como etiquetas en la propia ventana, cada una con su aspa para quitarlo.

5. Cómo se usa, día a día

Instalada y con los valores de fábrica, no hay nada que hacer: todo viene encendido y funciona sola. Estos son los tres casos en los que vas a abrir la ventana emergente.

Una página no funciona. Ábrela y pulsa **Permitir en este sitio**. La página se recarga sin filtrar. Si con eso se arregla, ya lo tienes resuelto para siempre en ese dominio. Si quieres afinar en vez de rendirte, vuelve a quitar el permiso y prueba apagando solo el **Filtro cosmético**: es el que rompe maquetaciones, mientras que el filtrado de red rara vez estorba.

Estás midiando tu propia web. Si trabajas con Analytics, Tag Manager o una herramienta de medición, tu propio bloqueador te está tapando lo que intentas ver. Apaga **EasyPrivacy** y **Consent Mode / Google Tags** mientras dure el trabajo, y vuelve a encenderlos al terminar.

Quieres apoyar a un canal. Entra en un vídeo suyo y pulsa el botón con el nombre del canal. A partir de ahí verás sus anuncios y el creador cobrará por ellos. El resto de YouTube sigue bloqueado.

6. Qué se ve en las páginas

En una página con publicidad, con la extensión puesta, no verás el anuncio **ni el hueco que deja**. Esa es la diferencia entre bloquear y ocultar bien: si solo se corta la descarga, queda un marco vacío que descuadra la columna.

Los *skins* y *wallpapers* —esas campañas que visten el fondo de la página entera con una marca— también desaparecen. La extensión busca imágenes de fondo servidas desde dominios publicitarios conocidos y las quita, dejando el fondo limpio.

En YouTube, el guion detecta el anuncio, pulsa **Saltar** cuando aparece, y si no hay botón adelanta el vídeo hasta el final del anuncio y lo silencia mientras tanto. También oculta el banner superior y los anuncios de la barra lateral. En Disney+, Prime Video y HBO Max hace lo equivalente en cada plataforma.

7. Preguntas frecuentes y problemas

¿Por qué los contadores muestran un guion? Porque Firefox no lo permite. Contar bloqueos en Manifest V3 se hace con una función llamada `declarativeNetRequest.getMatchedRules`, y **Firefox no la ha implementado**: solo existe si el usuario activa a mano una preferencia interna del navegador. Lo confirma el validador oficial de Mozilla, que marca esa llamada como no soportada. En la versión de Chrome el contador sí funciona.

Se decidió mostrar un guion en vez de un cero porque un cero permanente parece una avería, y no lo es: el bloqueo está funcionando, lo único que no se puede es contarlo. Pasando el ratón por encima sale la explicación.

Una tienda no me deja pagar / el banco no carga. Pulsa **Permitir en este sitio** y recarga. Se comprobaron las 29.740 reglas contra una lista de dominios sensibles (PayPal, Stripe, Redsys, Adyen, Braintree, Klarna,

reCAPTCHA, hCaptcha y las CDN de librerías) y **ninguna regla bloquea una pasarela de pago, un captcha ni una CDN**, pero una tienda puede servir una pieza propia desde un dominio que sí esté en la lista.

Una página se ve descuadrada. Apaga el **Filtro cosmético**. Es la capa que oculta elementos, y por tanto la única que puede afectar a cómo se ve una página. Si con eso se arregla, ya sabes a quién culpar.

He quitado el permiso de acceso a los sitios y ha dejado de ocultar anuncios. Es lo esperado. En Firefox los permisos de acceso a sitios son revocables, y sin ellos el navegador no inyecta los guiones de contenido, que son los que hacen el filtro cosmético. El filtrado de red sigue funcionando porque no necesita ese permiso. Para recuperarlo: `about:addons` → Ecom AdBlocker → **Permisos**.

YouTube ha dejado de saltar anuncios. YouTube cambia su código con frecuencia y detecta bloqueadores. Cuando lo cambia, hay que revisar los selectores del guion. Es inherente a esta clase de extensión, le pasa a todos.

¿Puedo añadir más reglas de bloqueo? No sin quitar otras. Firefox garantiza 30.000 reglas estáticas activas y la extensión usa 29.740: quedan **260 de margen**. Añadir a una lista obliga a recortar otra.

¿Recoge algún dato mío? No. No hay ni una llamada a Internet en todo el código, ni analítica, ni servidores propios. El manifest lo declara formalmente ante Mozilla con `data_collection_permissions: none`. Las preferencias se guardan en tu navegador y no salen de ahí.

¿Por qué el validador de Mozilla avisa de «coin miners»? Es un falso positivo, y conviene saberlo antes de subirla a addons.mozilla.org. El fichero `rules/easyprivacy.json` contiene las palabras `coinhive`, `coin-hive`, `coinimp` y `monero` porque son **dominios que la extensión bloquea**. El validador ve esas palabras en un fichero y avisa. Es justo lo contrario de ejecutar un minero.

8. Seguridad

Se ha revisado el `manifest.json`, el proceso de fondo, los cinco guiones de contenido, la ventana emergente y las seis listas de reglas (29.740 reglas). La revisión se cerró con el **validador oficial de Mozilla** (`addons-linter`) en **cero errores**, y con un banco de pruebas en navegador que pulsa todos los botones y verifica el filtro cosmético sobre los datos reales: **0 fallos de 32 comprobaciones**.

Esta extensión comparte código con la versión de Chrome, revisada justo antes. Se comprobó fallo por fallo cuáles se repetían aquí —los datos son idénticos byte a byte— y cuáles no tenían sentido en Firefox.

Lo que se ha arreglado

1. Inyección de CSS en las páginas visitadas, y filtro cosmético caído en 55 dominios

`content/cosmetic.js`, función `applyExtFilters()`.

El código construía la hoja de estilos concatenando en una sola cadena todos los selectores y añadiendo `{display:none!important;}` al final. En `cosmetic_filters.json` hay **84 entradas que no son selectores sino reglas de CSS completas**, con llaves incluidas (`body {padding-top:0;}` y parecidas), repartidas por **55 dominios**.

Se reprodujo en el navegador con una lista mínima. Con las entradas `['.ADBAR', '.otra-cosa', 'body {padding-top:0;}'], ['.tercera']`, el código original produce **una sola regla**:

```
selector:      .ADBAR, .otra-cosa, body
declaración: padding-top: 0px;
```

Es decir, tres cosas a la vez: la llave **cierra la regla antes de tiempo** y aplica `padding-top:0` a la página (`body` pasó de 99 px a 0 px en la prueba), el `display:none` que debía ocultar el anuncio **desaparece**, y todo lo que venía detrás de la entrada defectuosa — `.tercera` — **se pierde**. Sobre los datos reales el efecto es que en esos 55 dominios se caían también los **13.910 selectores genéricos**: el filtro cosmético no hacía absolutamente nada.

Arreglo: se descartan las entradas con llaves, comentarios de CSS o pseudoclases de Adblock Plus (`:has-text`, `:-abp-`, `:xpath`, `:style( ...)`), y las reglas se insertan con `CSSStyleSheet.insertRule()` por lotes de 100, reintentando uno a uno si un lote falla, de modo que ningún selector defectuoso pueda tumbar a los demás.

Verificado reintroduciendo el fallo, sobre `hindustantimes.com`, que es uno de los 55:

	Código original	Arreglado
Reglas CSS creadas	1	140
Selectores que actúan	13.927, pero con la declaración equivocada	13.926
¿Se oculta el anuncio?	NO	SÍ
¿Se ve el contenido real?	Sí	Sí

2. Guion de contenido que podía morir entero al arrancar `content/cosmetic.js`, funciones `injectStyle()` y `applyExtFilters()`.

Los guiones se inyectan en `document_start`, y en ese momento el documento puede no tener todavía ni la etiqueta `<html>`. El código hacía `(document.head || document.documentElement).appendChild(...)`, que sobre un valor nulo lanza una excepción y **se lleva por delante el guion de contenido completo**, filtro cosmético incluido, sin dejar rastro visible en la página. Salí al montar el banco de pruebas, que reproduce ese instante exacto.

Arreglo: se comprueba que exista dónde insertar y, si no, se reintentará en el siguiente ciclo (hasta 100 veces) en lugar de reventar. Verificado: con el código anterior el banco de pruebas da `Cannot read properties of null (reading 'appendChild')` y cero selectores; con el arreglo, 13.926 selectores activos.

3. Una entrada mal escrita en la lista blanca la anulaba entera `background/service_worker.js`, función `applyWhitelist()`.

Los dominios se pasaban tal cual a `initiatorDomains`, que solo admite dominios en minúsculas, sin puerto, sin ruta y sin punto inicial. Un único valor inválido guardado en los ajustes hacía que la llamada lanzara una excepción, y la lista blanca **completa dejaba de aplicarse en silencio**: el usuario vería sus sitios permitidos en la lista y seguirían filtrándose.

Arreglo: función `cleanDomains()` que normaliza (minúsculas, quita puerto y ruta), comprueba contra una expresión regular y descarta duplicados y basura.

4. Mensajes internos sin comprobar el remitente `background/service_worker.js` y `content/youtube.js`.

Los receptores de `runtime.onMessage` atendían cualquier mensaje sin mirar quién lo enviaba, y el proceso de fondo usaba el identificador de pestaña recibido sin validarlo.

Arreglo: se comprueba `sender.id === chrome.runtime.id` en los tres receptores y que el identificador de pestaña sea un entero no negativo.

5. Permiso `<all_urls>` innecesario, y ventana emergente que se quedaba sin dominio `manifest.json` y `popup/popup.js`.

La extensión pedía `host_permissions: ["<all_urls>"]`. Se comprobó que **no le hacía falta**: las 29.740 reglas son todas de tipo `block` (29.683) o `allow` (57), y con el permiso `declarativeNetRequest` esas dos acciones no necesitan permisos de host —los necesitarían `redirect` o `modifyHeaders`, que no se usan, ni hay ninguna regla con expresión regular—. El acceso a las páginas para el filtro cosmético ya lo pide `content_scripts`.

Además, en Firefox los permisos de host de Manifest V3 son **revocables por el usuario**, así que la ventana emergente podía quedarse sin saber en qué dominio está y el botón de lista blanca sin funcionar.

Arreglo: fuera `host_permissions`; en su lugar `activeTab`, que da acceso a la dirección de la pestaña **solo al pulsar el icono y solo a esa pestaña**. Como red de seguridad, si aun así no llegara la dirección, la ventana se la pide al guion de contenido con un mensaje `getHost` (responde solo el marco principal, y solo a mensajes de la propia extensión). Verificado: con dominio sale `elmundo.es` y el botón funciona; en `about:addons` sale «página interna» y el botón queda desactivado, que es lo correcto.

6. Contador que en Firefox iba a quedarse en cero para siempre `background/service_worker.js` y `popup/popup.js`.

El contador usa `declarativeNetRequest.getMatchedRules`, que **Firefox no implementa** —lo marca el validador oficial de Mozilla como `UNSUPPORTED_API`—. Además, el manifest ni siquiera declaraba el permiso `declarativeNetRequestFeedback` que esa función exige. El resultado era una ventana que enseñaba «0 en esta pestaña» y «0 total bloqueados» de forma permanente, que es exactamente lo que parece una extensión averiada.

Arreglo: se detecta si la función existe; si no, se ahorra el trabajo inútil y la ventana muestra **un guion** con una explicación al pasar el ratón. No se ha añadido el permiso `declarativeNetRequestFeedback` a propósito: pediría un permiso más al usuario para algo que en Firefox no va a funcionar igualmente.

7. Pie de la ventana que mentía con el bloqueo apagado `popup/popup.js`. Con el interruptor general en «off», el pie seguía diciendo «110 reglas de red activas» cuando en realidad no queda ninguna. *Arreglo:* ahora dice «**Bloqueo desactivado**».

8. Interruptor de streaming que no existía Los guiones de Disney+, Prime Video y HBO Max consultaban una preferencia `streaming` que **no estaba en los valores por defecto ni tenía ningún interruptor**: no había forma de desactivarlos. *Arreglo:* añadida a los valores por defecto y añadido su interruptor en la ventana emergente.

9. Manifest incompatible con su propio mínimo de Firefox `manifest.json` .

Declaraba `strict_min_version: 128.0` y a la vez usaba `data_collection_permissions` , que Firefox no entiende hasta la 140 (142 en Android). El validador de Mozilla lo marcaba dos veces. *Arreglo*: mínimo subido a 140.0, y añadido `gecko_android` con 142.0. Las dos advertencias desaparecen.

10. Versión y documentación desincronizadas La ventana emergente mostraba «v1.1.0» escrita a mano con el manifest en 1.3.0. *Arreglo*: la versión se lee de `runtime.getManifest()` . Además, el `README.md` era el de **Chrome**, con la versión equivocada y cifras de reglas que no coincidían con las listas reales (decía 20.000 de EasyList y 8.000 de EasyPrivacy cuando son 16.000 y 6.000, y no mencionaba dos de las seis listas ni los tres guiones de *streaming*). Se ha rehecho entero con los datos comprobados.

11. Nueve megabytes de compilaciones viejas dentro del paquete La carpeta `web-ext-artifacts/` guardaba **15 paquetes antiguos (9,1 MB)**, desde la 1.1.0 hasta la 1.3.0. *Arreglo*: movidos a `modulos IA/utiles/ecom_adblocker-firefox/web-ext-artifacts/` . La extensión pasa de 15 MB a **5,8 MB**.

12. Ortografía Corregidas «pestaña» → «pestaña», «cosmetico» → «cosmético» y «pagina interna» → «página interna» en la ventana emergente, y la tilde de la descripción del manifest. El revisor deja la carpeta en **TOTAL: 0**.

Lo que del arreglo de Chrome NO se ha traído, y por qué

`use_dynamic_url` para que las webs no detecten la extensión. En Chrome se añadió esa clave a `web_accessible_resources` para que la dirección del fichero de filtros rote y una página no pueda comprobar si lo tienes instalado. **En Firefox esa clave no existe** y añadirla no haría nada. No hace falta: Firefox sirve los recursos de las extensiones desde `moz-extension://<UUID>/` , y ese UUID es **aleatorio y distinto en cada instalación**, precisamente para impedir esa huella. La protección la da el navegador de serie. Se deja como está, documentado.

El permiso `declarativeNetRequestFeedback` . En Chrome es lo que hace funcionar el contador. Firefox reconoce el permiso pero no la función que lo usa, así que añadirlo solo serviría para pedirle al usuario un permiso más a cambio de nada.

Lo que se ha revisado y estaba bien

- **Sin código dinámico**: no hay `eval()` , `new Function()` , `document.write()` ni `insertAdjacentHTML` . El único `innerHTML` de todo el proyecto es un `= ''` para vaciar un contenedor.
- **Sin telemetría ni servidores propios**: cero llamadas a Internet. No hay claves de API ni credenciales en el código.
- **Sin permisos peligrosos**: no usa `cookies` , `webRequest` , `history` , `bookmarks` , `downloads` , `management` , `debugger` , `scripting` ni `tabs` .
- **Sin puerta de entrada externa**: no hay `onMessageExternal` ni `externally_connectable` , así que ninguna página web puede hablar con la extensión.
- **La ventana emergente escapa correctamente**: los nombres de canal y los dominios se pintan con `textContent` , nunca con `innerHTML` , de modo que un nombre de canal con HTML dentro no puede ejecutar nada.
- **Las reglas de red no rompen tiendas**: se cruzaron las 29.740 reglas contra una lista de dominios sensibles (PayPal, Stripe, Redsys, Adyen, Braintree, Klarna, Bizum, reCAPTCHA, hCaptcha, cdnjs, jsDelivr,

Google Fonts y bancos españoles). Las coincidencias son todas legítimas: complementos de jQuery para servir anuncios, píxeles de analítica de Shopify y Magento, y dominios de anuncios detrás de Cloudflare.

Ninguna regla bloquea una pasarela de pago, un captcha ni una CDN de librerías.

- `background.scripts` **es lo correcto en Firefox**, que no implementa `background.service_worker`. La clave `persistent` no está puesta, que es lo que corresponde en Manifest V3.
- **Los topes de Firefox están comprobados:** 6 listas declaradas de 100 posibles, 6 activas de 20, 1 regla dinámica de 5.000 y 0 reglas con expresión regular de 1.000.

Lo que queda pendiente

- **Margen de reglas casi agotado.** Las 29.740 reglas estáticas están a **260** del tope de 30.000 que garantiza Firefox. Añadir reglas obligará a quitarlas de otra lista, o a repartirlas y rotarlas. No es un fallo, pero condiciona cualquier ampliación.
- **La lista blanca no cubre los `iframe` de terceros.** Se aplica según el dominio del marco que hace la petición, así que un `iframe` ajeno dentro de un sitio permitido sigue filtrándose. Corregirlo cambia cómo cada marco decide si actúa, y eso cambia el comportamiento del producto: se deja documentado y sin tocar.
- **Las 84 entradas defectuosas siguen en `cosmetic_filters.json`.** El código ya las descarta sin consecuencias, pero el origen está en el conversor de listas (`utils/ecom_adblocker/tools/convert.js`), que traduce mal los filtros `:style()` de EasyList. Conviene arreglarlo **antes** de regenerar las listas.
- **El código usa `chrome.*` con funciones de retorno.** Funciona —la 1.3.0 está firmada y publicada, y el validador no lo señala—, pero la documentación de Mozilla solo garantiza explícitamente el estilo de promesas para `chrome.*` en Manifest V3. Pasarlo a `browser.*` sería más ortodoxo; es un cambio grande, en todos los ficheros, y no se ha tocado por no arriesgar sin poder probarlo en Firefox de verdad.
- **No se ha podido probar en Firefox real.** En esta máquina solo hay Chromium para Playwright, y Playwright no permite cargar extensiones en Firefox. Todo lo verificable sin Firefox se ha verificado: el validador oficial de Mozilla, el banco de pruebas en navegador y el análisis de las listas. **Queda por hacer una pasada manual en Firefox** antes de publicar.
- **Los guiones de vídeo dependen de los selectores de cada plataforma.** YouTube, Disney+, Prime Video y HBO Max cambian su código con frecuencia. Es inherente a esta clase de extensión.

9. Ficha técnica

Estructura

```
ecom_adblocker-firefox/
  manifest.json           Manifest V3, Firefox 140+ / Android 142+
  README.md
  background/service_worker.js  event page: listas, lista blanca, contador
  content/cosmetic.js         ocultación de anuncios + borrado de fondos
  content/cosmetic_filters.json 13.910 selectores genéricos + 9.165 dominios
  content/youtube.js         anti-anuncios YouTube + lista blanca de canales
  content/disneyplus.js
  content/primevideo.js
  content/hbomax.js
  rules/network_rules.json    110 reglas propias
  rules/easylist.json         16.000
  rules/easylist_es.json      617
  rules/easyprivacy.json      6.000
  rules/google_tags.json      13
  rules/extra_domains.json    7.000
  popup/popup.html · popup.css · popup.js
  icons/icon16.png · icon48.png · icon128.png
```

Tamaño del paquete: **5,8 MB**.

Permisos y por qué

Permiso	Para qué
<code>declarativeNetRequest</code>	Aplicar las 29.740 reglas de bloqueo
<code>storage</code>	Guardar las preferencias en el navegador
<code>activeTab</code>	Leer la dirección de la pestaña al pulsar el icono
<code>content_scripts</code> con <code><all_urls></code>	Injectar el filtro cosmético

No se usan `host_permissions`, ni `tabs`, ni `declarativeNetRequestFeedback`.

Guiones de contenido

Fichero	Dónde se inyecta	Marcos
<code>cosmetic.js</code>	Todas las páginas	Todos
<code>youtube.js</code>	<code>*.youtube.com</code>	Solo el principal
<code>hbomax.js</code>	<code>*.hbomax.com</code> , <code>*.max.com</code>	Solo el principal
<code>primevideo.js</code>	<code>*.primevideo.com</code> , <code>*.amazon.com</code> , <code>*.amazon.es</code>	Todos
<code>disneyplus.js</code>	<code>*.disneyplus.com</code>	Todos

Todos con `run_at: document_start`.

Mensajes internos

Mensaje	De → a	Para qué
<code>getTabCount</code>	ventana emergente → fondo	Pedir el contador (devuelve <code>supported: false</code> en Firefox)
<code>getHost</code>	ventana emergente → <code>cosmetic.js</code>	Respaldo para saber el dominio de la pestaña
<code>ytGetChannel</code>	ventana emergente → <code>youtube.js</code>	Canal e identificador para el botón de lista blanca

Los tres receptores comprueban `sender.id` antes de responder.

Claves de almacenamiento

`enabled` · `lists` · `whitelist` · `ytWhitelist` · `totalBlocked`. Todas en `storage.local`. Valores por defecto: bloqueo activado, las nueve listas encendidas, listas blancas vacías, contador a cero.

Reglas dinámicas

Una sola, con identificador 100000 y prioridad 3, de tipo `allow`, cuyo `initiatorDomains` es la lista blanca ya saneada. Se reescribe entera cada vez que cambia la lista blanca.

Validación

```
npx addons-linter@latest .
```

0 errores, 0 avisos de manifest, 3 avisos (dos por `getMatchedRules` y uno de `COINMINER_USAGE_DETECTED` sobre `rules/easyprivacy.json`, falso positivo por los dominios de minería que la extensión bloquea).

Banco de pruebas

`modulos IA/utiles/ecom_adblocker-firefox/probar.py` — 32 comprobaciones en navegador: abre la ventana emergente, exige cero errores de JavaScript, **pulsa los nueve interruptores y los dos botones** verificando que cada uno escribe de verdad, comprueba el caso del contador no disponible y el de la pestaña sin dominio, y ejecuta el filtro cosmético real sobre `cosmetic_filters.json` completo en uno de los 55 dominios afectados. Resultado actual: **FALLAN: 0**.

`modulos IA/utiles/ecom_adblocker-firefox/antes_despues.py` — reintroduce el fallo del CSS para demostrar que la prueba lo caza.

Registros

La extensión no escribe ficheros de registro. Los mensajes de diagnóstico van a la consola del proceso de fondo, que se abre en `about:debugging#/runtime/this-firefox` → **Inspeccionar** sobre Ecom AdBlocker.